

VLSI Design Internship – Task 3 Report

Verilog RTL Design of Sequential Circuits and Flip-Flops

1. Introduction to Sequential Circuits

Sequential circuits differ from combinational circuits because their output depends on both the current inputs and the previous state. They use memory elements called flip-flops and are driven by a clock signal. Sequential circuits are the foundation of processors, controllers, and memory systems.

2. Tools Used

- Vivado (Xilinx) – for writing, compiling, and simulating Verilog code
- Built-in waveform viewer – for analysing simulation output

3. D Flip-Flop

The D Flip-Flop captures the value of input D at every rising clock edge and transfers it to output Q. It is the simplest and most widely used flip-flop.

Module Code:

```
module d_flipflop(  
    input clk,  
    input D,  
    output reg Q  
);  
always @(posedge clk)  
begin  
    Q <= D;  
end  
endmodule
```

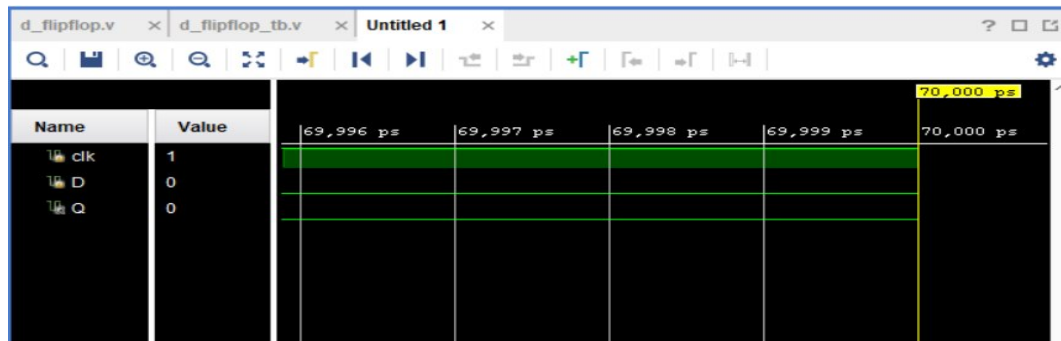
Testbench:

```
module tb_d_flipflop;  
    reg clk, D;  
    wire Q;  
    d_flipflop uut (.clk(clk), .D(D), .Q(Q));  
    always #5 clk = ~clk;  
    initial begin  
        clk = 0; D = 0;  
        #10 D = 1;  
        #10 D = 0;  
        #10 D = 1;  
        #10 D = 1;  
        #10 D = 0;  
        #20 $finish;  
    end  
endmodule
```

Truth Table – D Flip-Flop:

Clock Edge	D	Q (Next State)
Rising Edge	0	0
Rising Edge	1	1

Simulation Waveform:



4. JK Flip-Flop

The JK Flip-Flop has two inputs J and K. It can Set, Reset, hold, or Toggle the output depending on the input combination. When J=1 and K=1 it toggles, which resolves the invalid state problem of the SR flip-flop.

Module Code:

```
module jk_flipflop(
    input clk,
    input J,
    input K,
    output reg Q
);
    always @(posedge clk)
    begin
        case({J,K})
            2'b00: Q <= Q; // No change
            2'b01: Q <= 0; // Reset
            2'b10: Q <= 1; // Set
            2'b11: Q <= ~Q; // Toggle
        endcase
    end
endmodule
```

Testbench:

```

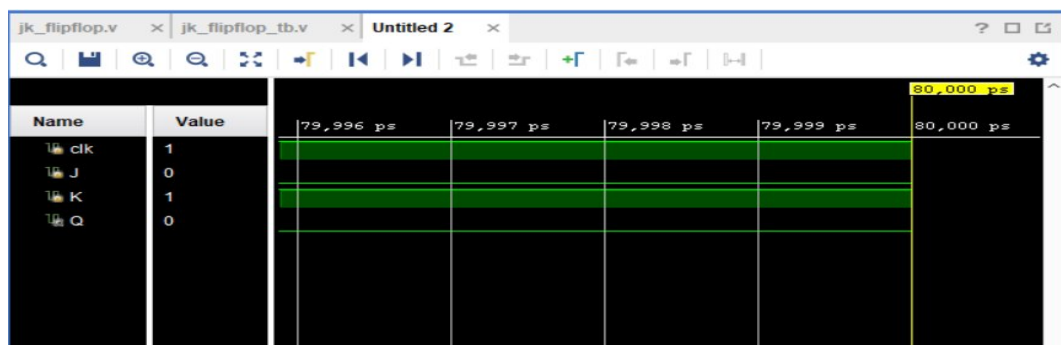
module tb_jk_flipflop;
reg clk, J, K;
wire Q;
jk_flipflop uut (.clk(clk), .J(J), .K(K), .Q(Q));
always #5 clk = ~clk;
initial begin
clk = 0; J = 0; K = 0;
#10 J = 0; K = 0;
#10 J = 0; K = 1;
#10 J = 1; K = 0;
#10 J = 1; K = 1;
#10 J = 1; K = 1;
#10 J = 0; K = 1;
#20 $finish;
end
endmodule

```

Truth Table – JK Flip-Flop:

J	K	Q (Next State)
0	0	No change (Q)
0	1	Reset (0)
1	0	Set (1)
1	1	Toggle (~Q)

Simulation Waveform:



5. 4-bit Register

A 4-bit register stores 4 bits of data simultaneously. It consists of four D flip-flops sharing a common clock. On every rising clock edge, the 4-bit input D is loaded into the register and appears at output Q.

Module Code:

```

module register4(
input clk,
input [3:0] D,
output reg [3:0] Q
);
always @(posedge clk)
begin
Q <= D;
end
endmodule

```

Testbench:

```

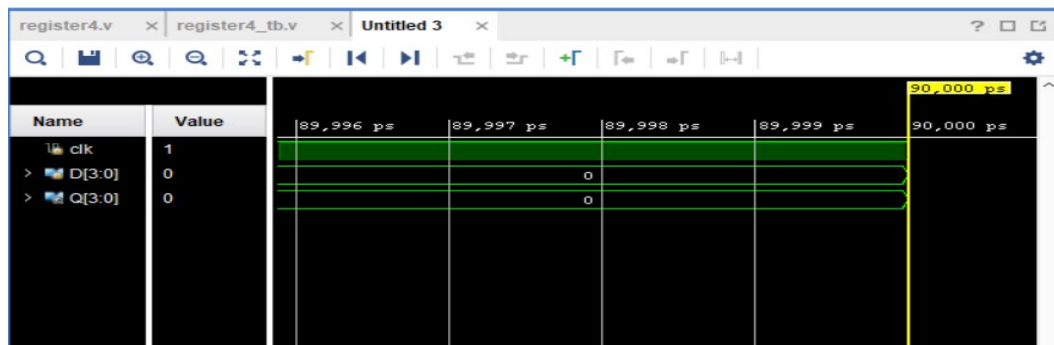
module tb_register4;
reg clk;
reg [3:0] D;
wire [3:0] Q;
register4 uut (.clk(clk), .D(D), .Q(Q));
always #5 clk = ~clk;
initial begin
clk = 0; D = 4b0000;
#10 D = 4'b0001;
#10 D = 4'b0011;
#10 D = 4'b0111;
#10 D = 4'b1111;
#10 D = 4'b1010;
#10 D = 4'b0000;
#20 $finish;
end
endmodule

```

Truth Table – 4-bit Register:

Clock Edge	D [3:0]	Q [3:0]
Rising Edge	0000	0000
Rising Edge	0001	0001
Rising Edge	0111	0111
Rising Edge	1111	1111
Rising Edge	1010	1010

Simulation Waveform:



6. 4-bit Binary Counter

A 4-bit binary counter increments its count value by 1 on every rising clock edge. It counts from 0 to 15 (0000 to 1111 in binary) and then wraps back to 0. A synchronous reset input forces the count back to 0 when asserted.

Module Code:

```
module counter4(
    input clk,
    input reset,
    output reg [3:0] count
);
    always @(posedge clk)
    begin
        if(reset)
            count <= 4'b0000;
        else
            count <= count + 1;
        end
    endmodule
```

Testbench:

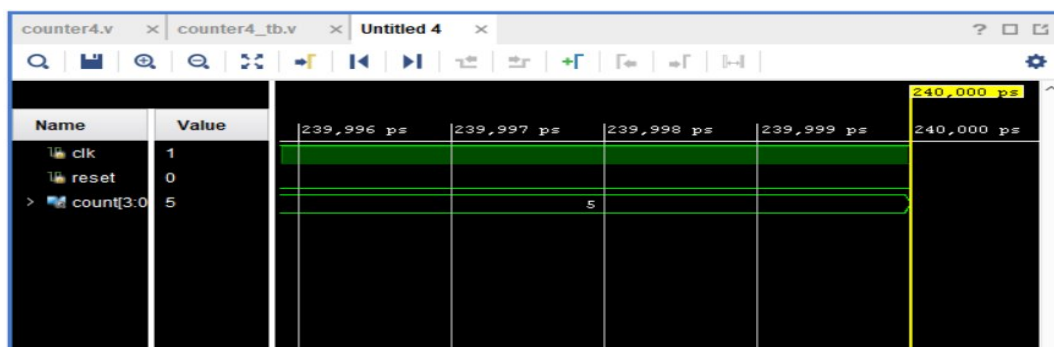
```
module tb_counter4;
    reg clk, reset;
    wire [3:0] count;
    counter4 uut (.clk(clk), .reset(reset), .count(count));
    always #5 clk = ~clk;
    initial begin
        clk = 0; reset = 1;
        #10 reset = 0;
        #160;
        #10 reset = 1;
        #10 reset = 0;
        #50 $finish;
    end
endmodule
```

Truth Table – 4-bit Counter (partial):

Clock Cycle	Reset	count [3:0]
-------------	-------	-------------

1	1	0000
2	0	0001
3	0	0010
4	0	0011
5	0	0100
6	0	0101
...	0	...
16	0	1111
17	0	0000 (wrap)

Simulation Waveform:



7. Testbench (Verification)

All sequential circuits were verified using testbench files in Vivado. A clock signal was generated using always #5 clk = ~clk to toggle every 5 ns (10 ns clock period). Input stimulus was applied in the initial block and outputs were observed in the waveform viewer. Outputs were verified against expected truth tables for each circuit.

8. Observations

- D Flip-Flop captures input D only at the rising clock edge — output Q does not change between clock edges.
- JK Flip-Flop correctly shows No Change (J=0,K=0), Reset (J=0,K=1), Set (J=1,K=0), and Toggle (J=1,K=1) behaviour.
- 4-bit Register loads all 4 input bits simultaneously on the rising clock edge and holds the value until the next clock.
- 4-bit Counter increments by 1 on each rising clock edge, counts from 0 to 15, and resets to 0 when reset=1.
- All sequential circuits use the always @(posedge clk) block which makes them synchronous — they only respond to the rising clock edge.
- Non-blocking assignment (<=) is used in all sequential modules, which is the correct RTL coding practice for flip-flop based circuits.